



The Relational Model

Logical View of Data

Keys

Integrity Rules

Relational Model



Relational Model represents how data is stored in Relational Databases. A relational database stores data in the form of relations (tables).

Objectives

- To promote high degree of data independence
- Eliminate redundancy and consistency problems.
- Ease of use of DBA as well as normal users.
- To provide a community view of the data of spartan simplicity, so that a wide variety of users can interact with a common model.

The main use of Relational model is data storage and processing.

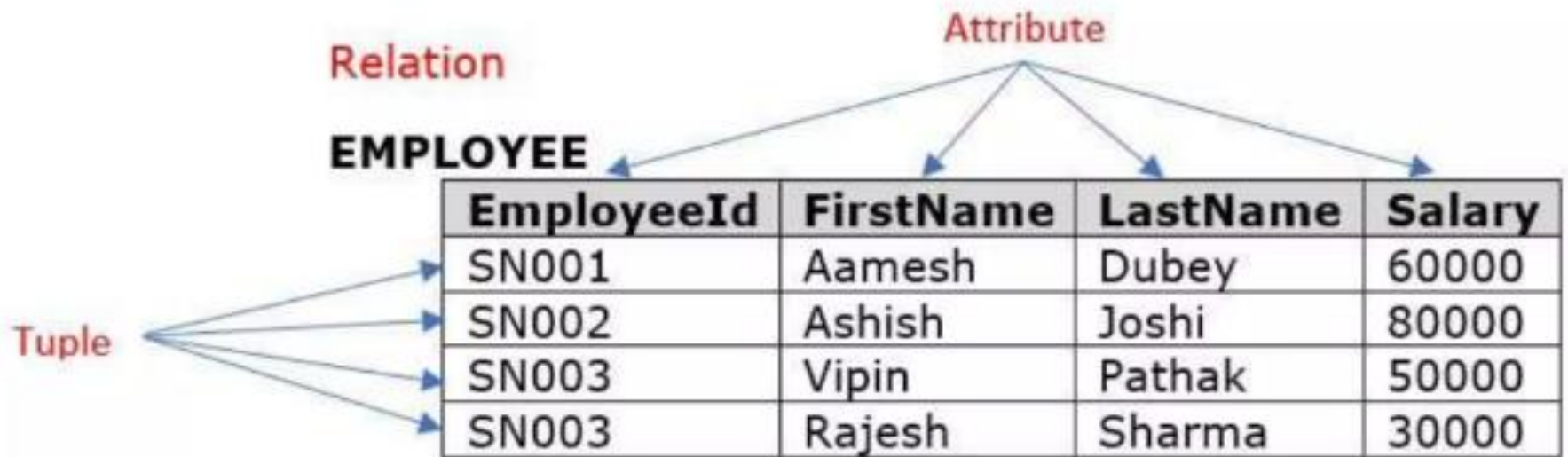
Relational Database



A Relational database is a database that organizes data into tables which can be linked—or related—based on data common to each.

Relational Database Components

1. **Relation:** A relation is nothing but a table with rows and columns.
2. **Attributes:** Attributes are the properties that define a relation.
3. **Tuple:** Each row in the relation is known as tuple.
4. **Domain:** A domain is a unique set of values permitted for an attribute in a table.
5. **Degree:** The number of attributes in a relation.
6. **Cardinality:** The number of tuples in a relation.



Relation Example

relation

attributes

tuples

Product ID	Product Name	Supplier	Category	Quantity Per Unit	Unit Price	Units In Stock
1	Chai	1	1	10 boxes x 20 bags	\$18.00	39
2	Chang	1	1	24 - 12 oz bottles	\$19.00	17
3	Aniseed Syrup	1	2	12 - 550 ml bottles	\$10.00	13
4	Chef Anton's Cajun Seasoning	2	2	48 - 8 oz jars	\$22.00	53
5	Chef Anton's Gumbo Mix	2	2	36 boxes	\$21.35	0
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	\$25.00	120
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	\$30.00	15
8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	\$40.00	6
9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	\$97.00	29
10	Ikura	4	8	12 - 200 ml jars	\$31.00	31
11	Queso Cabrales	5	4	1 kg pkg.	\$21.00	22
12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	\$20.00	20

Degree = 7
Cardinality = 77

Domain of Unit Price is *currency*.



Properties of Relations

- Relation name is distinct from all other relation names in relational schema.
- Each cell of relation contains exactly one atomic (single) value.
- Each attribute has a distinct name.
- Values of an attribute are all from the same domain.
- Each tuple is distinct; there are no duplicate tuples.
- Order of attributes has no significance.
- Order of tuples has no significance, theoretically.
- Key – every table must have a unique key to identify each row



Relational Keys



- **Key**

A key is an attribute or set of attributes which helps us in uniquely identifying the rows of a table. It also helps in establishing relationship among tables.

Types

- Super Key
- Candidate Key
- Primary Key
- Foreign Key
- Composite Key

Primary Key

Primary Key is a column/ that uniquely identify every row in that table. The Primary Key can't be a duplicate meaning the same value can't appear more than once in the table. A table cannot have more than one primary key.



Candidate Key



Candidate Key is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes. The Primary key should be selected from the candidate keys. Every table must have at least a single candidate key.



Alternate/Secondary Key

All the candidate key which are ***not a primary key*** are called an alternate key.



Super Key



A **Super key** or *simply key* is a combination of all possible attribute which can uniquely identify the rows(tuples) in a table. This means that a super key may have some extra attribute which isn't necessary for uniquely identifying the rows in the table.

Student

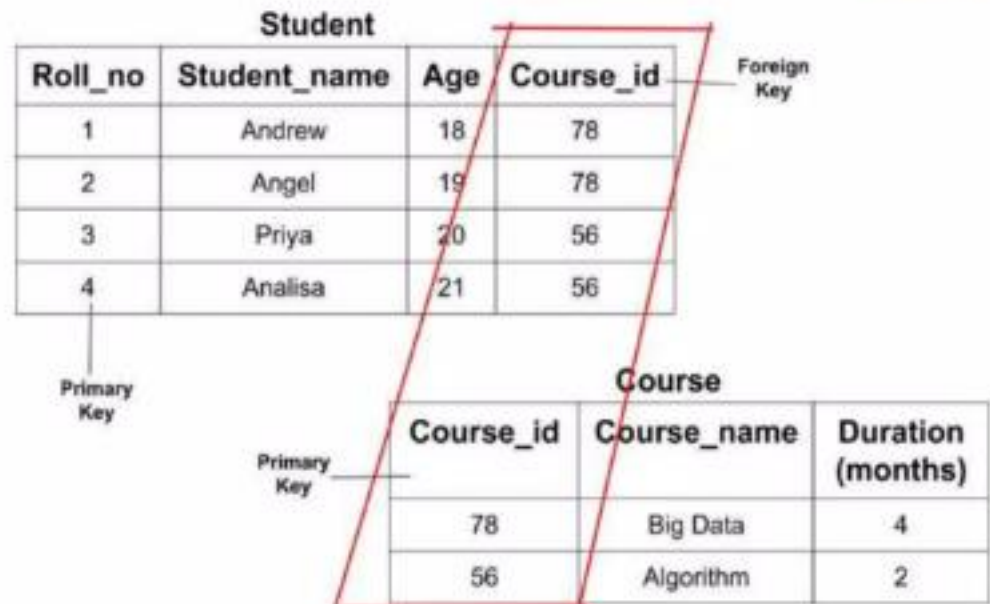
Roll_no	Name	Registration_no
1	Andrew	895
2	Angel	564
3	Augusto	567

1. {Roll_no}
2. {Registration_no}
3. {Roll_no, Registration_no},
4. {Roll_no, Name}
5. {Name, Registration_no}
6. {Roll_no, Name, Registration_no}

Foreign Key



Foreign Key is a column/attribute that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity. It acts as a cross-reference between two tables as it references the primary key of another table.



Composite Key



A key that has more than one attributes is known as composite key. It is also known as compound key.

Any key such as **super key**, **primary key**, **candidate key** etc. can be called composite key if it has more than one attributes.

Composite Key (cont.)



cust_Id	order_Id	product_code	product_count
-----	-----	-----	-----
C01	0001	P007	23
C02	0123	P007	19
C02	0123	P230	82
C01	0001	P890	42

None of these columns **alone** can play a role of key in this table.

Key in above table: {cust_id, product_code}

This is a composite key as it is made up of more than one attributes.

Relation Schema



A relation schema represents name of the relation with its attributes.

So, A set of table attributes is called a relation schema. For that reason, a relation schema is also known as table schema. A relation schema can be thought of as the basic information describing a table or relation. Hence, it is the logical definition of a table.

e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE and AGE) is a relation schema for STUDENT.

If a schema has more than 1 relation, it is called **Relational Schema aka Database Schema**. You could say that a database schema is made up of lots of relation schema and shows how they work together.



Integrity Rules

Integrity rules refer to reliability, validity and authority towards the data. They needed to inform the DBMS about certain constraints in the real world.

There are two principal rules:

1. Entity integrity &
2. Referential integrity



Entity Integrity Rule



For Entity Integrity Rule, each table has a Primary Key and the Primary Key can not have a Null value.

<Student>

Student_ID	Student_Name	Student_Awards
------------	--------------	----------------

Above, you can see our primary key is **Student_ID**. We cannot consider Student_Awards as the primary key since not every student would have received the award.

<Employee>

Employee_ID	Employee_Name	Employee_Age	Employee_Location
-------------	---------------	--------------	-------------------

In the above table, **Employee_ID** is the primary key.

Entity Integrity (cont.)



So let us now summarize the Entity Integrity Rule –

- i. Make sure that each tuple in a table is unique.
- ii. Every table must have a primary key, for example, **Student_ID** for a Student table.
- iii. Every entity is unique.
- iv. The relations Primary Key must have unique values for each row.
- v. Primary Key cannot have NULL value and must be unique.
- vi. Example can be an **Employee_ID** cannot be null in an Employee table

Referential Integrity Rule



Referential Integrity Rule in DBMS is based on Primary and Foreign Key. The Rule defines that a foreign key have a matching primary key. Reference from a table to another table should be valid.

EMP_ID	EMP_NAME	DEPT_ID
E01	Tom	D003
E02	David	D004
E03	Brian	D002

DEPT_ID	DEPT_NAME	DEPT_ZONE
D001	Finance	North
D002	HR	South
D003	Marketing	East
D004	Technical	West

Primary Key

Foreign Key

The rule states that the **DEPT_ID** in the Employee table has a matching valid **DEPT_ID** in the **Department** table. To allow join, the referential integrity rule states that the Primary Key and Foreign Key have same data types.

General Constraints



These are additional rules specified by users or database administrators that define or constraint some aspect of the enterprise.

They are alsoknown as business rules.

For example :

The Cust_No in Customer table is given a range from 0001-5000.

Keys in RDBMS

- ☆ Uniquely identify the tuple.
- ☆ Super key.
- ☆ Candidate key.
- ☆ Primary key.
- ☆ Alternate key.
- ☆ Unique key.
- ☆ Composite key.
- ☆ Foreign key.

Employee					
ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Super key

- ☆ Like **superset**.
- ☆ **Uniquely** identify the tuple.
- ☆ NULL values.
- ☆ {**Name**} is not a super key.
- ☆ May contain **extraneous attributes**.
- ☆ Superkeys:
 - {**ID**}, {**SSN**}, {**ID, Name**},
 - {**ID, SSN**}, {**ID, Phone**},
 - {**Name, Phone**}, {**ID, Email**},
 - {**Name, SSN, Phone**},
 - {**Name, Email**},

ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk



Candidate key

☆ Superkeys:

{ID}, {SSN}, {ID, Name},
{ID, SSN}, {ID, Phone},
{Name, Phone}, {ID, Email},
{Name, SSN, Phone},
{Name, Email},
{ID, SSN, Phone}

☆ Minimal super keys are called candidate keys.

☆ Candidate Keys:

{ID}, {SSN}, {Name, Phone},
{Email}

Employee					
ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Primary key selected from candidate key, where, it satisfies the constraint -> **which can be unique, not contain null values,**

Candidate Keys : eg

- {ID},
- {SSN-every country – not unique}
- Other having – null values

So, chosen Primary key – {ID}

- Primary key chosen by DBA
- Never or very rarely changed
- Candidate key with null values is not the primary key

Alternate key:

The candidate other than the primary key is called as alternate keys

Not primary keys

Unique Keys:

Choose a Unique values but may have null values

Unique:

100% unique but support null values {email}, {phone number},
{Name, Phone number}

Unique key

☆ Candidate Keys:

{ID}, {SSN}, {Name, Phone},
{Email}

☆ Primary Key: {ID}

☆ Alternate Keys:

{SSN}, {Name, Phone}, {Email} ...

☆ Unique Key: {Name, Phone} ...

☆ Composite Key: {Name, Phone} ...



Employee					
ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Foreign key – deals with two tables

Referential integrity – if any values you are inserting it should be validated by another table

Foreign key

Student			
S_ID	Name	Dept_Code	Credits
101	John	101	12
102	Robin	102	14
103	Alya	103	20
104	Yusuf	104	10

Dept	
Dept_Code	Dept_Name
101	CSE
102	EEE
103	ECE
104	MECH



Referential Integrity